Name: **Jash Chauhan**

Section: **D – (D1)**

EXP: 5

Aim: To study and execute different joins in SQL.

Employees Table

```
CREATE TABLE Employees (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(50),
   dept_id INT,
   salary DECIMAL(10, 2),
   manager_id INT
);
```

Departments Table

```
CREATE TABLE Departments_x (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(50),
   location VARCHAR(50)
);
```

Projects Table

```
CREATE TABLE Projects (
   proj_id INT PRIMARY KEY,
   proj_name VARCHAR(50),
   dept_id INT
);
```

Assignments Table

```
CREATE TABLE Assignments (
```

```
    emp_id INT,
    proj_id INT,
    hours_worked INT,
    PRIMARY KEY (emp_id, proj_id),
    FOREIGN KEY (emp_id) REFERENCES Employees(emp_id),
    FOREIGN KEY (proj_id) REFERENCES Projects(proj_id)
);
```

Employees Table Data

```
INSERT INTO Employees (emp_id, emp_name, dept_id, salary, manager_id) VALUES (1, 'John', 101, 5000.00, NULL);
INSERT INTO Employees (emp_id, emp_name, dept_id, salary, manager_id) VALUES (2, 'Alice', 102, 3000.00, 1);
INSERT INTO Employees (emp_id, emp_name, dept_id, salary, manager_id) VALUES (3, 'Robert', 101, 4000.00, 1);
INSERT INTO Employees (emp_id, emp_name, dept_id, salary, manager_id) VALUES (4, 'Michael', 103, 7000.00, NULL);
INSERT INTO Employees (emp_id, emp_name, dept_id, salary, manager_id) VALUES (5, 'Susan', 102, 3500.00, 2);
INSERT INTO Employees (emp_id, emp_name, dept_id, salary, manager_id) VALUES (6, 'Nancy', 104, 4200.00, NULL);
INSERT INTO Employees (emp_id, emp_name, dept_id, salary, manager_id) VALUES (7, 'David', 103, 3900.00, 4);
INSERT INTO Employees (emp_id, emp_name, dept_id, salary, manager_id) VALUES (8, 'Laura', 101, 5200.00, 3);
```

Departments Table Data

```
INSERT INTO Departments_x (dept_id, dept_name, location) VALUES (101, 'Finance', 'New York');
INSERT INTO Departments_x (dept_id, dept_name, location) VALUES (102, 'HR', 'Chicago');
INSERT INTO Departments_x (dept_id, dept_name, location) VALUES (103, 'IT', 'Dallas');
INSERT INTO Departments_x (dept_id, dept_name, location) VALUES (104, 'Marketing', 'San Francisco');
```

Projects Table Data

INSERT INTO Projects (proj_id, proj_name, dept_id) VALUES (201, 'Project A', 101);

INSERT INTO Projects (proj_id, proj_name, dept_id) VALUES (202, 'Project B', 103);

INSERT INTO Projects (proj_id, proj_name, dept_id) VALUES (203, 'Project C', 102);

INSERT INTO Projects (proj_id, proj_name, dept_id) VALUES (204, 'Project D', 104);

Assignments Table Data

INSERT INTO Assignments (emp_id, proj_id, hours_worked) VALUES (1, 201, 30);

INSERT INTO Assignments (emp_id, proj_id, hours_worked) VALUES (2, 203, 40);

INSERT INTO Assignments (emp_id, proj_id, hours_worked) VALUES (3, 201, 20);

INSERT INTO Assignments (emp_id, proj_id, hours_worked) VALUES (4, 202, 50);

INSERT INTO Assignments (emp_id, proj_id, hours_worked) VALUES (5, 203, 35);

INSERT INTO Assignments (emp_id, proj_id, hours_worked) VALUES (6, 204, 25);

INSERT INTO Assignments (emp_id, proj_id, hours_worked) VALUES (7, 202, 45);

1. INNER JOIN

Objective: Retrieve and display the list of employees along with their department names.

- ⇨ **SELECT e.emp_name, d.dept_name FROM Employees e INNER JOIN Departments_x d ON e.dept_id = d.dept_id;**

2. LEFT JOIN

Objective: Display all employees and their assigned projects, including employees who are not assigned to any project.

- ⇨ **SELECT e.emp_name, p.proj_name FROM Employees e LEFT JOIN Assignments a ON e.emp_id = a.emp_id LEFT JOIN Projects p ON a.proj_id = p.proj_id;**

3. RIGHT JOIN

Objective: List all projects and the employees assigned to them, including projects with no employees.

- **SELECT p.proj_name, e.emp_name FROM Projects p RIGHT JOIN Assignments a ON p.proj_id = a.proj_id RIGHT JOIN Employees e ON a.emp_id = e.emp_id;**

4. FULL OUTER JOIN

Objective: Get a list of all employees and departments, showing departments without employees and employees without departments.

- **SELECT e.emp_name, d.dept_name FROM Employees e FULL OUTER JOIN Departments_x d ON e.dept_id = d.dept_id;**

5. SELF JOIN

Objective: Display the list of employees along with their managers' names.

- **SELECT e.emp_name AS Employee, m.emp_name AS Manager FROM Employees e LEFT JOIN Employees m ON e.manager_id = m.emp_id;**

6. CROSS JOIN

Objective: Generate a report showing all possible pairs of employees and projects (regardless of assignment).

- **SELECT e.emp_name, p.proj_name FROM Employees e CROSS JOIN Projects p;**